

The Perceptron & Multilayer Perceptron

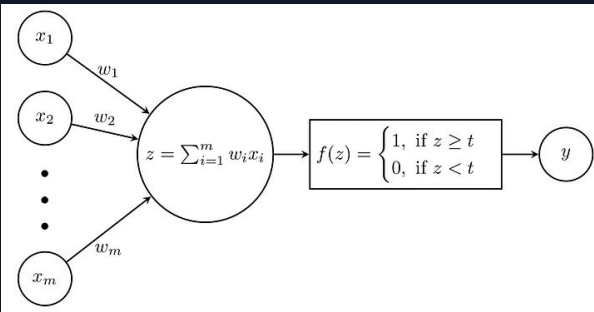
The beginnings of Neural Networks

Prof. Didi Ramsaran Chin

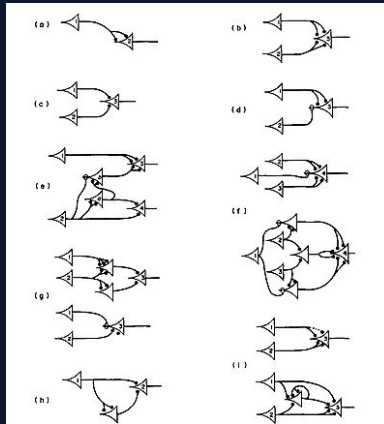
INFO-02020

October 2026

Timeline of the perceptron

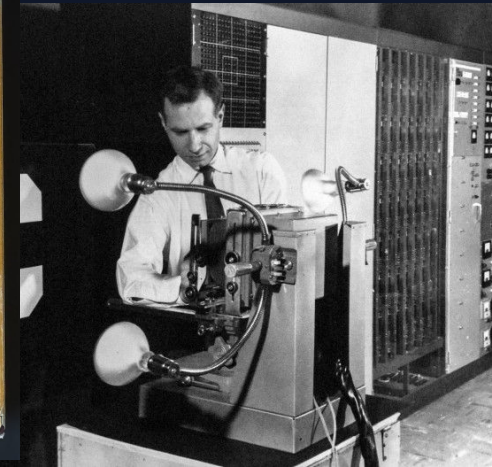
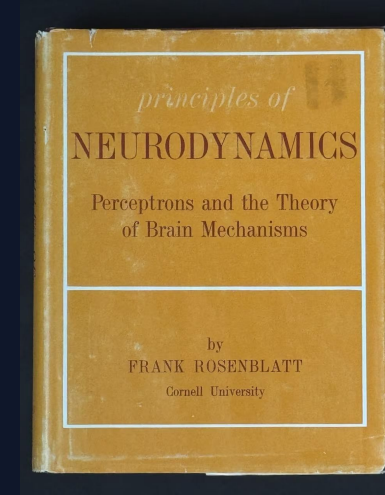


[Source](#)



1939–1952 - the squid

Hodgkin and Huxley combined biology and physics to create a circuit-like model (and its equations) of action potential



[Source: Wikimedia Commons](#)

1943 - Cambridge

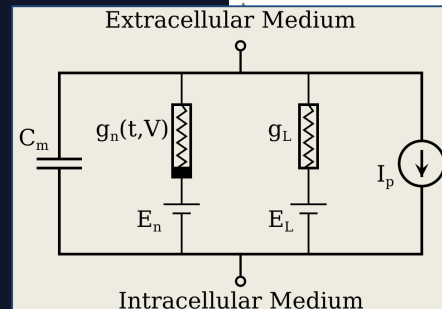
McCulloch and Pitts established the relation between neurons and boolean logic.

A QUANTITATIVE DESCRIPTION OF MEMBRANE CURRENT AND ITS APPLICATION TO CONDUCTION AND EXCITATION IN NERVE

By A. L. HODGKIN AND A. F. HUXLEY

From the Physiological Laboratory, University of Cambridge

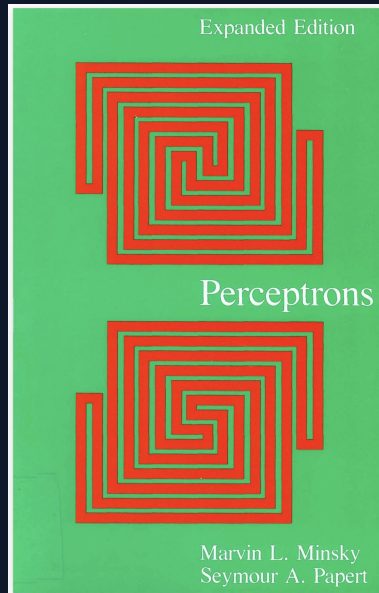
(Received 10 March 1952)



Rosenblatt's improvements to the Perceptron

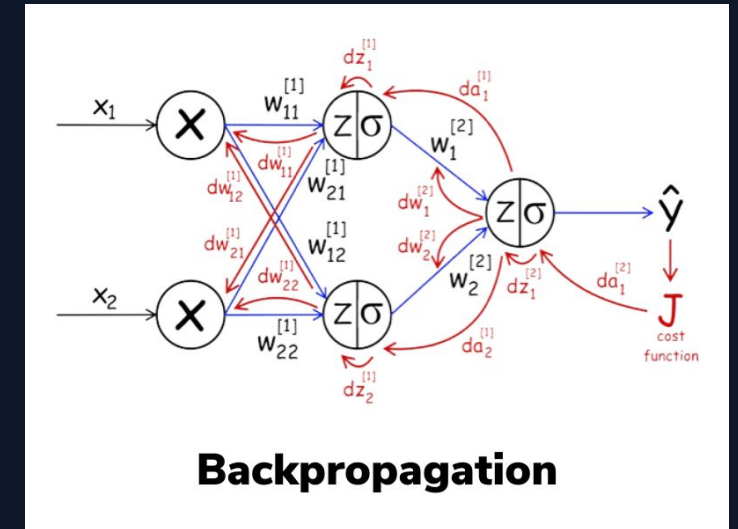
He added the model's bias, improved weights and developed a learning algorithm (1958-1962).

Timeline of the perceptron



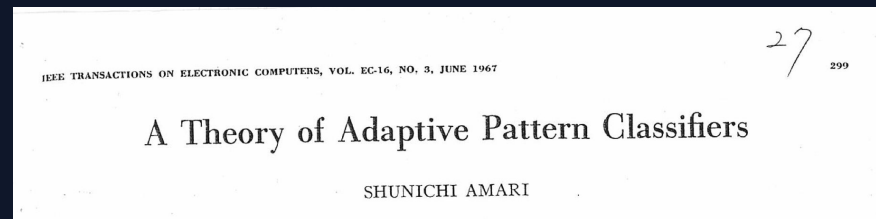
1967 - First MLP

Shunichi Amari proposed the first
MLP based on stochastic
gradient descent



1969 - Minsky & Papert

Fundamental limits of
single-layer perceptrons



1986 - Learning and backprop

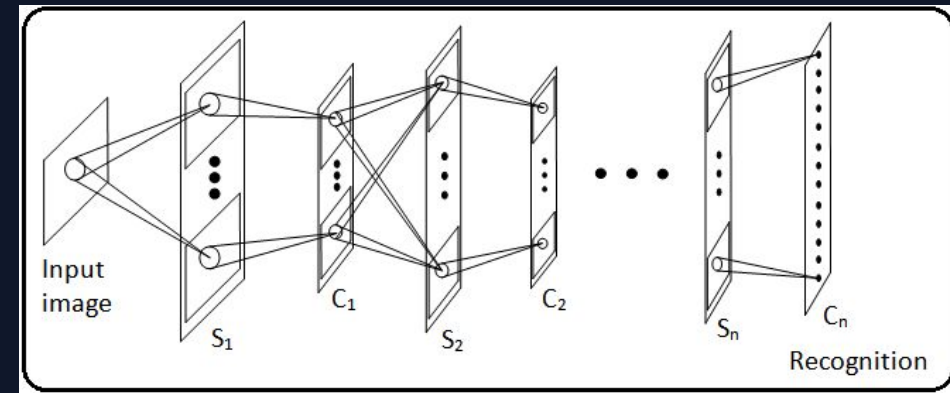
Rumelhart, Williams, Hinton
popularized backprop as a
training algorithm MLPs

The Evolution of AI

1982-1984 -

Associative Memories

Applied "strong" physics (dynamics, attractors) to develop recurrent memories capable of content-addressable memories



Proc. Natl. Acad. Sci. USA
Vol. 79, pp. 2554-2558, April 1982
Biophysics

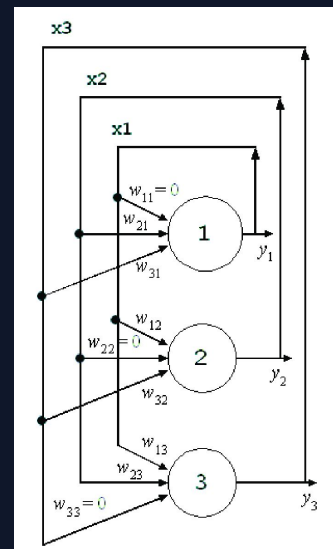
Neural networks and physical systems with emergent collective computational abilities

(associative memory/parallel processing/categorization/content-addressable memory/fail-soft devices)

J. J. HOPFIELD

Division of Chemistry and Biology, California Institute of Technology, Pasadena, California 91125; and Bell Laboratories, Murray Hill, New Jersey 07974

Contributed by John J. Hopfield, January 15, 1982

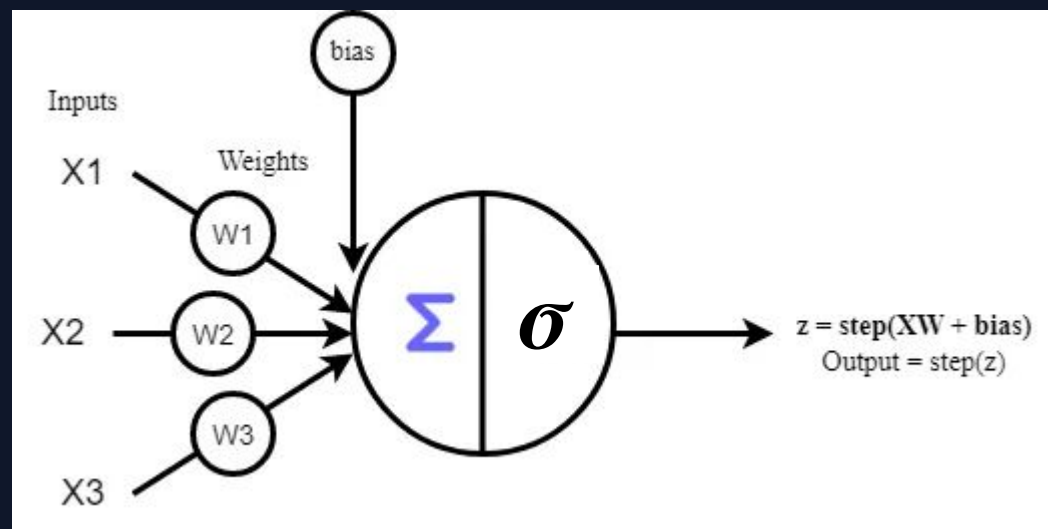


1980- The neocognitron

Frist CNN for machine vision

Remember: Rosenblatt's perceptron

Rosenblatt's



- Bias term added
- Parameters can be updated (!!)
- Outputs can be continuous
(activation function σ)

Weighing and summation (forward pass)

$$z = \sum_{i=1}^n w_i x_i + b$$

$$z = \mathbf{w}^T \mathbf{x} + b$$

Learning = adjusting parameters

$$w_i \leftarrow w_i + \alpha(y - \hat{y})x_i$$

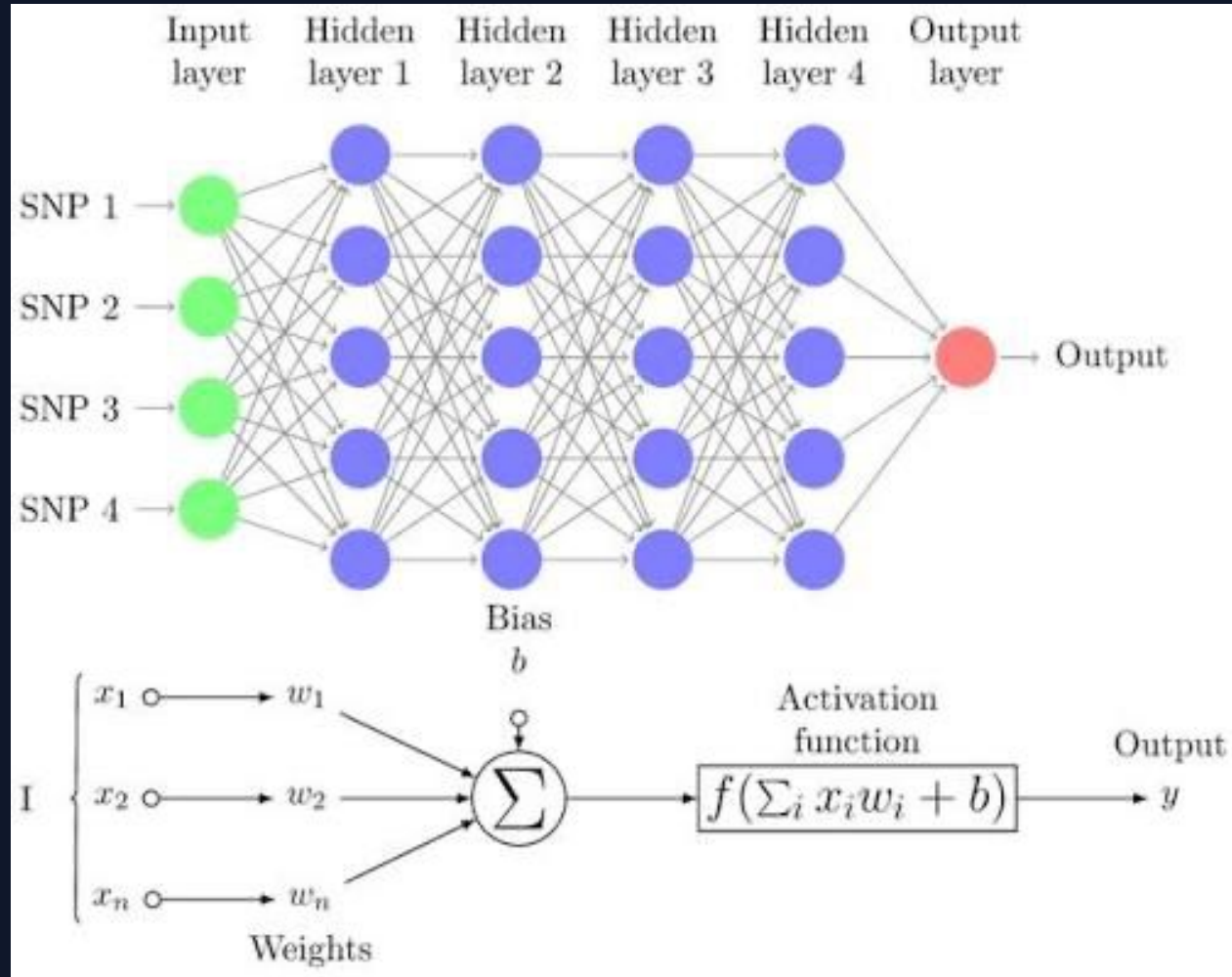
$$\mathbf{w} \leftarrow \mathbf{w} + \alpha(y - \hat{y})\mathbf{x}$$

$$b \leftarrow b + \alpha(y - \hat{y})$$

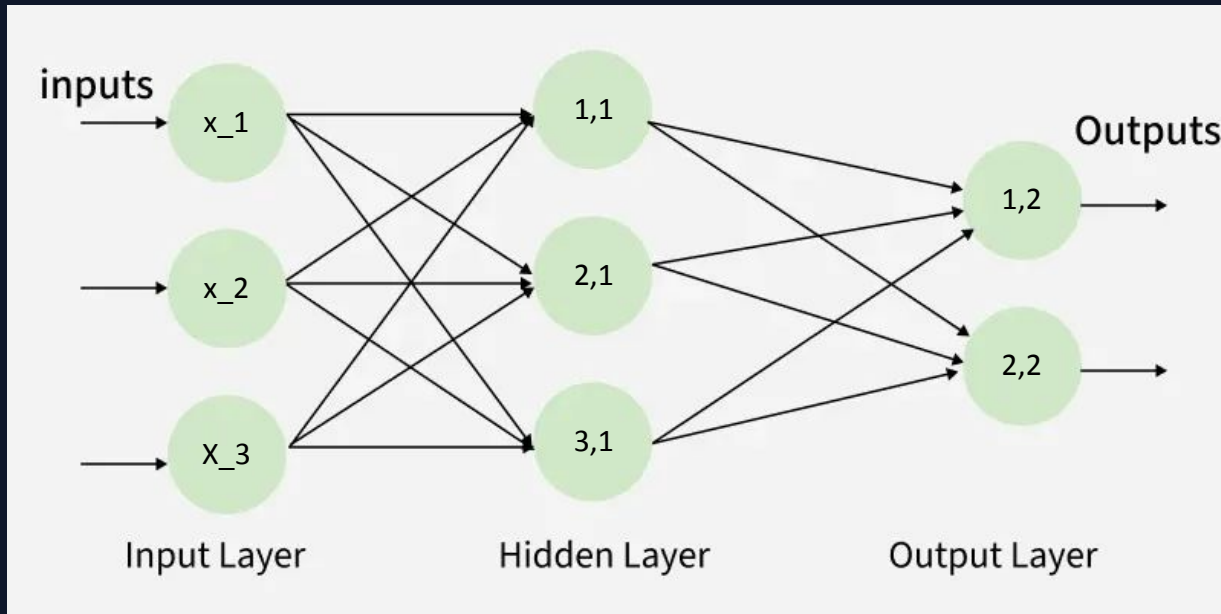
Recall

$$\hat{y} = \sigma(z)$$

Multilayer perceptron - the architecture



Multilayer perceptron - feedforward algorithm



Input — Hidden

$$h_{1,1}(x_1, x_2, x_3)$$

$$h_{2,1}(x_1, x_2, x_3)$$

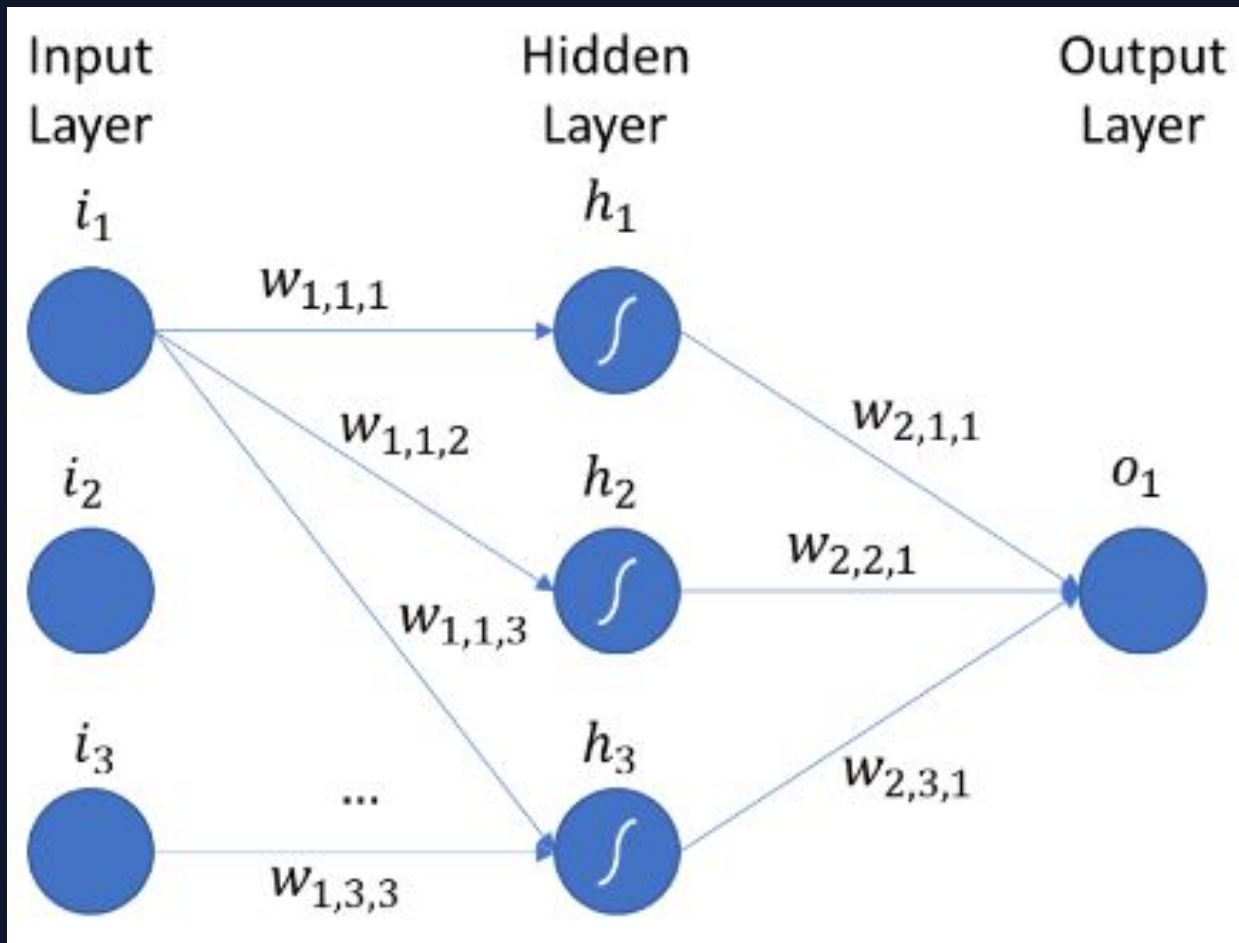
$$h_{3,1}(x_1, x_2, x_3)$$

Hidden — Output

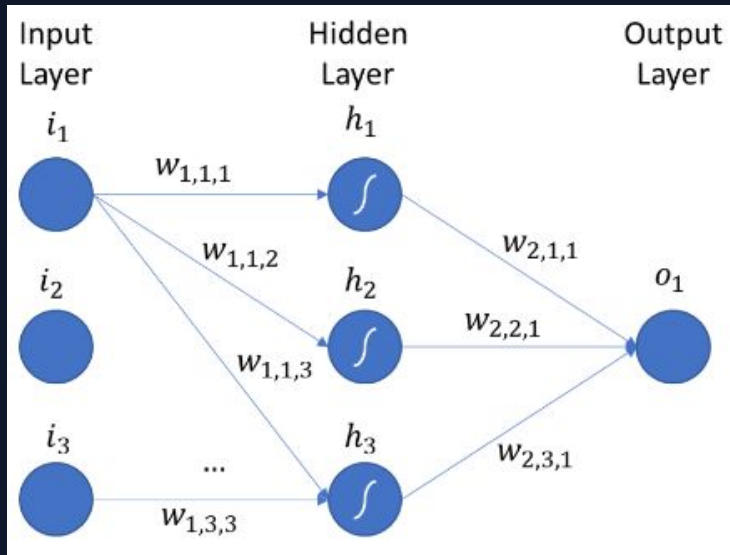
$$y_{1,2}(h_{1,1}; h_{2,1}; h_{3,1})$$

$$y_{2,2}(h_{1,1}; h_{2,1}; h_{3,1})$$

Multilayer perceptron - another diagram



Multilayer perceptron - backpropagation algorithm



Assume loss = $\text{MSE}(y_{\text{hat}}, y)$

Then the learning rule for the perceptron can be applied to the layer optimization process of output layer and weights associated

$$w_i \leftarrow w_i + \alpha(y - \hat{y})x_i$$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha(y - \hat{y})\mathbf{x}$$

Recall:

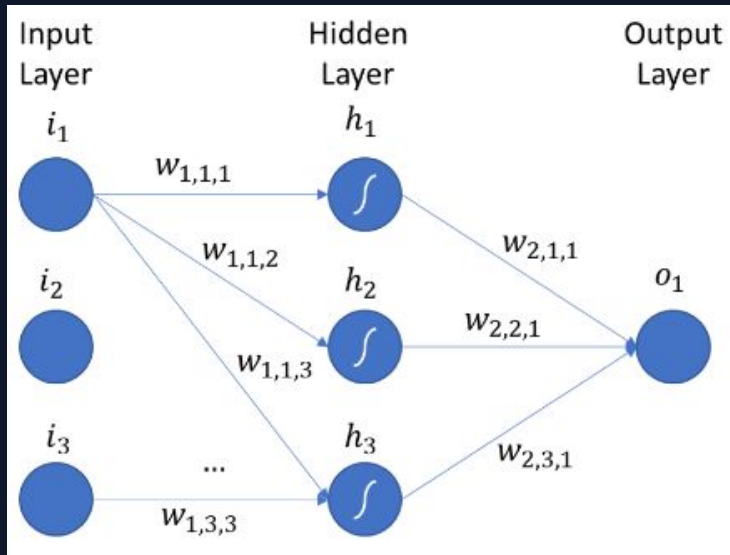
$$b \leftarrow b + \alpha(y - \hat{y})$$

Recall

$$\hat{y} = \sigma(z)$$

What about the other parameters?

Multilayer perceptron - backpropagation algorithm



We now need a better learning rule for our MLP:

$$\theta_{new} = \theta_{old} - \alpha \nabla_{\theta} L$$

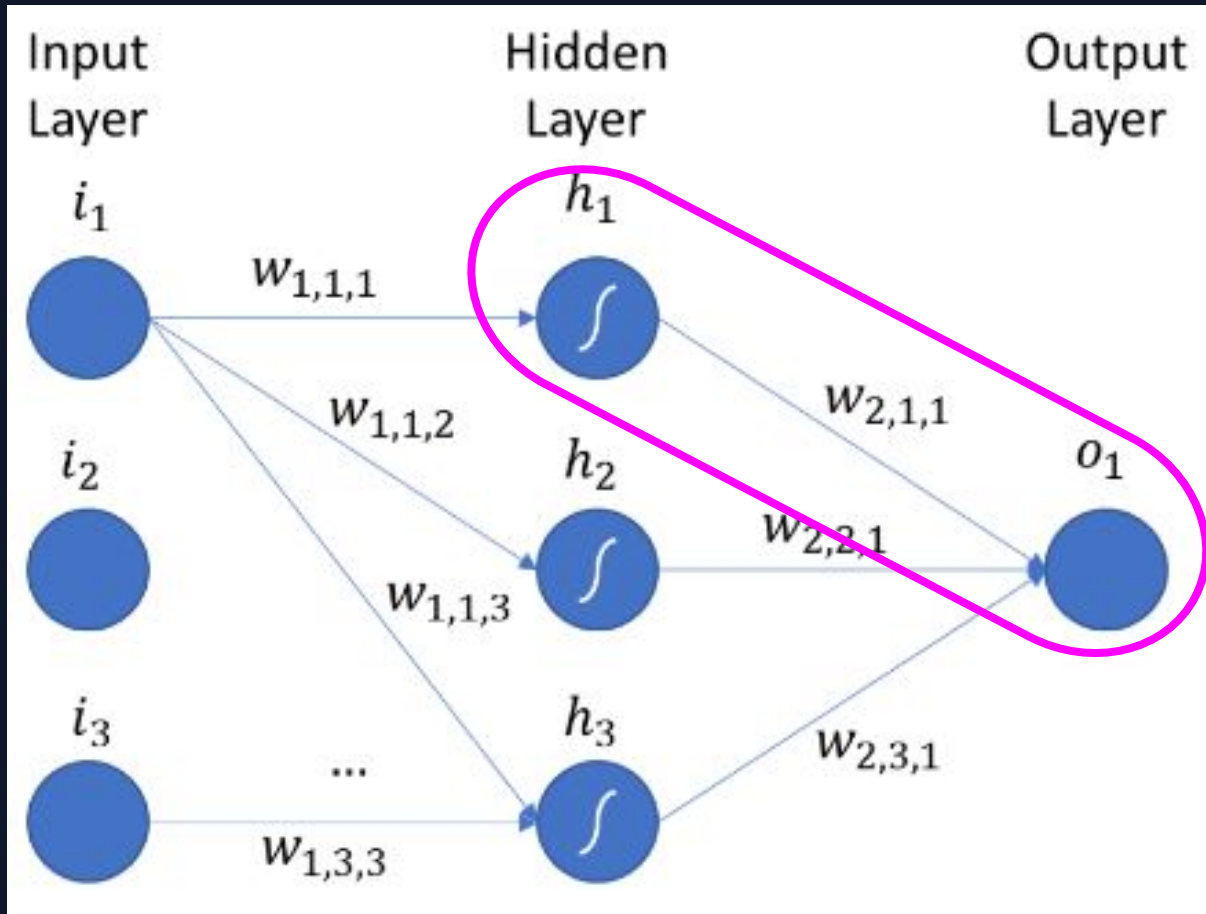
$$w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} Loss(\mathbf{w})$$

alpha = step size (hyperparameter)

nabla = gradient of loss with respect to theta

theta = parameter to be updated

Multilayer perceptron - backpropagation algorithm

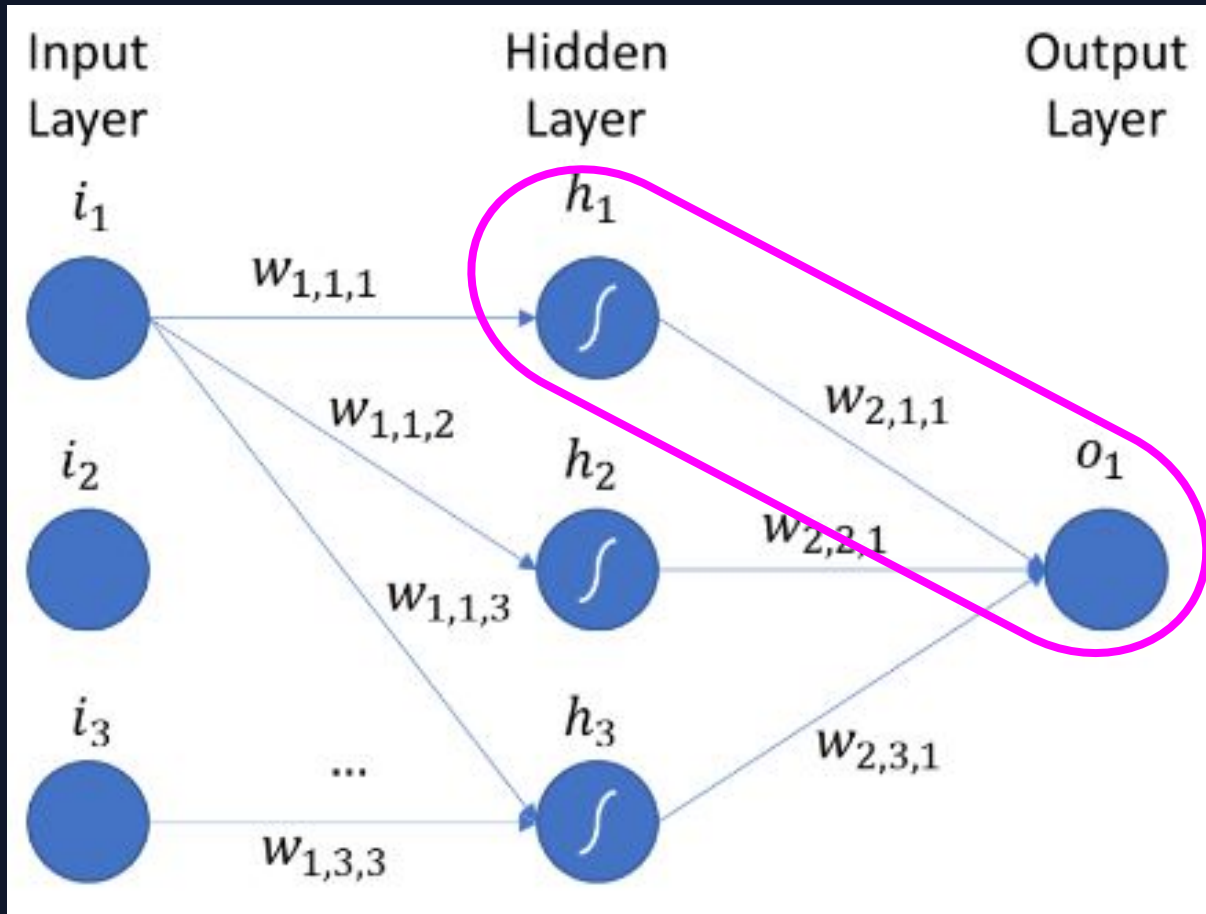


We need to adjust not only b_{output} and $w_{2,1}$, based on $\text{MSE}(\hat{y}, y)$

We need to adjust: b_{h1} and $w_{1,1}$, based on $\text{MSE}(\hat{y}, y)$ as well.

How?

Multilayer perceptron - backpropagation algorithm



- Pre-activation of h_1 :

$$z_{h1} = w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + w_{13}^{(1)}x_3 + b_{h1}$$

- Activation of h_1 :

$$h_1 = \sigma(z_{h1})$$

- Pre-activation of output:

$$z_{out} = w_1^{(2)}h_1 + w_2^{(2)}h_2 + w_3^{(2)}h_3 + b_{out}$$

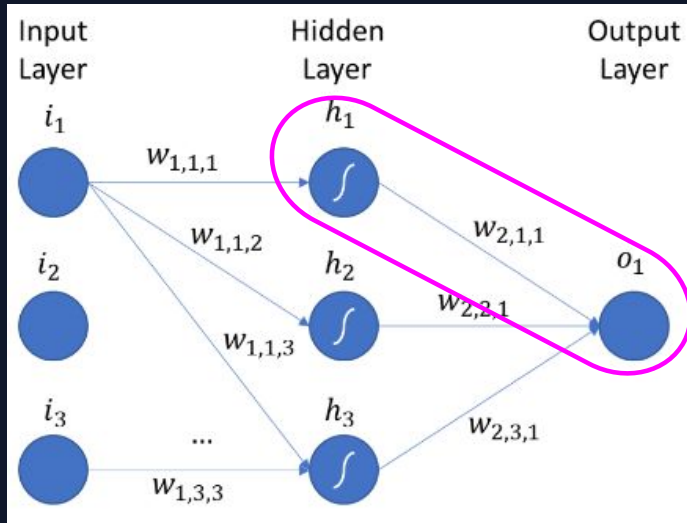
- Final Prediction:

$$\hat{y} = \sigma(z_{out})$$

- Loss:

$$L(y, \hat{y})$$

Multilayer perceptron - backpropagation algorithm



$$\delta_{out} = \frac{\partial L}{\partial z_{out}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_{out}}$$

activation func

loss function

$$\frac{\partial L}{\partial w_1^{(2)}} = \frac{\partial L}{\partial z_{out}} \cdot \frac{\partial z_{out}}{\partial w_1^{(2)}} = \frac{\partial L}{\partial w_1^{(2)}} = \delta_{out} \cdot h_1$$

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial z_{out}} \cdot \frac{\partial z_{out}}{\partial h_1} = \frac{\partial L}{\partial h_1} = \delta_{out} \cdot w_1^{(2)}$$

$$\delta_{h1} = \frac{\partial L}{\partial z_{h1}} = \frac{\partial L}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_{h1}} = \delta_{h1} = (\delta_{out} \cdot w_1^{(2)}) \cdot \sigma'(z_{h1})$$

- Pre-activation of h_1 :

$$z_{h1} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + w_{13}^{(1)} x_3 + b_{h1}$$

- Activation of h_1 :

$$h_1 = \sigma(z_{h1})$$

- Pre-activation of output:

$$z_{out} = w_1^{(2)} h_1 + w_2^{(2)} h_2 + w_3^{(2)} h_3 + b_{out}$$

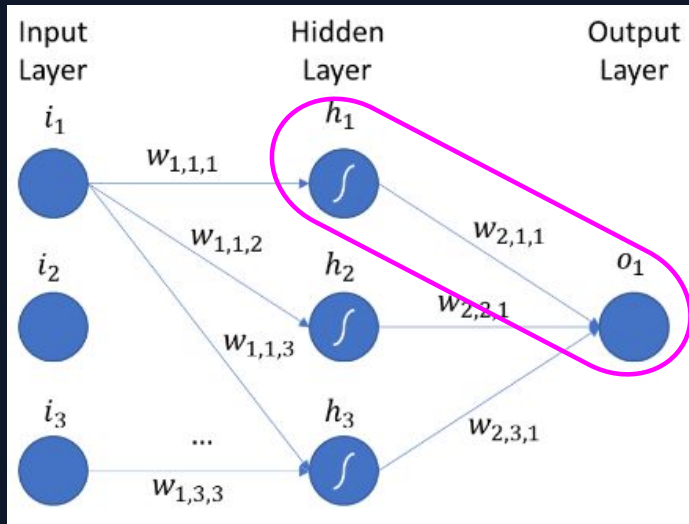
- Final Prediction:

$$\hat{y} = \sigma(z_{out})$$

- Loss:

$$L(y, \hat{y})$$

Multilayer perceptron - backpropagation algorithm



$$\delta_{out} = \frac{\partial L}{\partial z_{out}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_{out}}$$

activation func

loss function

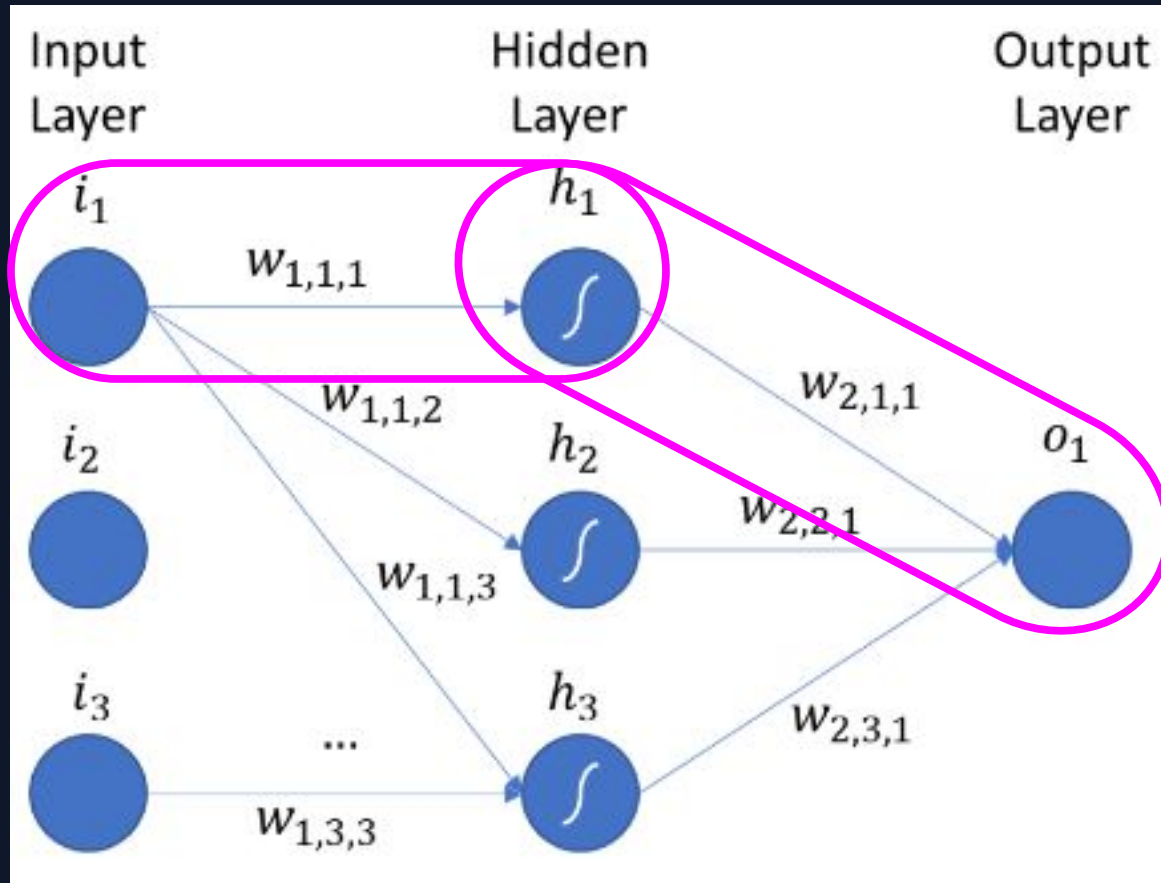
$$\frac{\partial L}{\partial w_1^{(2)}} = \frac{\partial L}{\partial z_{out}} \cdot \frac{\partial z_{out}}{\partial w_1^{(2)}} = \frac{\partial L}{\partial w_1^{(2)}} = \delta_{out} \cdot h_1$$

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial z_{out}} \cdot \frac{\partial z_{out}}{\partial h_1} = \frac{\partial L}{\partial h_1} = \delta_{out} \cdot w_1^{(2)}$$

$$\delta_{h1} = \frac{\partial L}{\partial z_{h1}} = \frac{\partial L}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_{h1}} = \delta_{h1} = (\delta_{out} \cdot w_1^{(2)}) \cdot \sigma'(z_{h1})$$

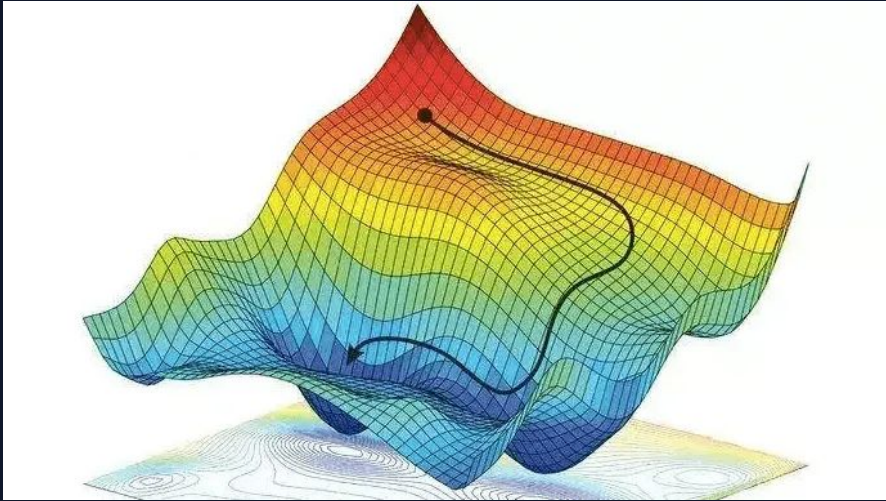
$$\frac{\partial L}{\partial w_{11}^{(1)}} = \frac{\partial L}{\partial z_{h1}} \cdot \frac{\partial z_{h1}}{\partial w_{11}^{(1)}} = \frac{\partial L}{\partial w_{11}^{(1)}} = \delta_{h1} \cdot x_1$$

Multilayer perceptron - backpropagation algorithm



$$\frac{\partial L}{\partial w_{11}^{(1)}} = \underbrace{\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_{out}}}_{\text{Output Error}} \cdot \underbrace{\frac{\partial z_{out}}{\partial h_1}}_{\text{Path Weight}} \cdot \underbrace{\frac{\partial h_1}{\partial z_{h1}}}_{\text{Hidden Activation}} \cdot \underbrace{\frac{\partial z_{h1}}{\partial w_{11}^{(1)}}}_{\text{Input}}$$

Recall - Gradient descent



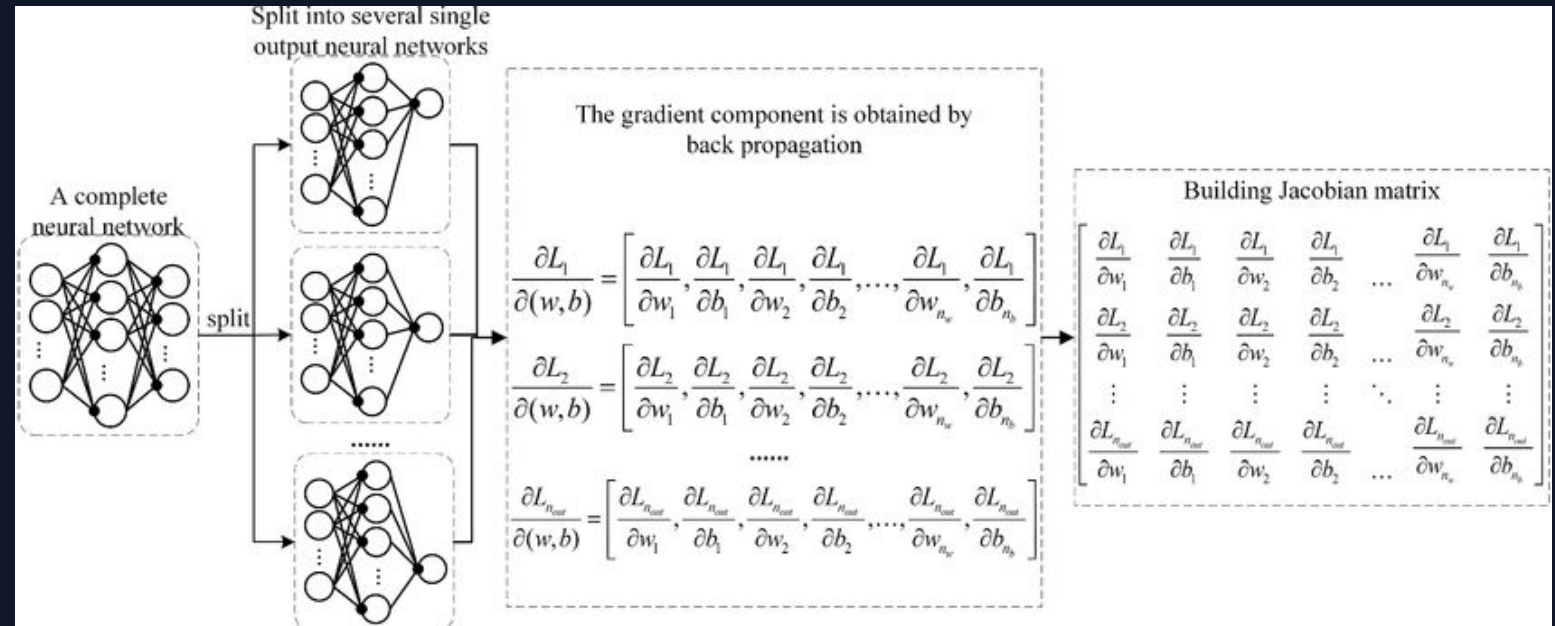
Source: <https://pub.aimind.so/gradient-descent-and-its-variants-0689c858652b>

$$\frac{\partial L}{\partial w_{11}^{(1)}} = \underbrace{\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_{out}}}_{\text{Output Error}} \cdot \underbrace{\frac{\partial z_{out}}{\partial h_1}}_{\text{Path Weight}} \cdot \underbrace{\frac{\partial h_1}{\partial z_{h1}}}_{\text{Hidden Activation}} \cdot \underbrace{\frac{\partial z_{h1}}{\partial w_{11}^{(1)}}}_{\text{Input}}$$

We use the chain rule to determine each component of the jacobian

$$\theta_{new} = \theta_{old} - \alpha \nabla_{\theta} L$$

$$w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Loss}(\mathbf{w})$$



Challenges of Machine Learning 4

Size of batch for gradient descent:

- **Batch GD:** all training data taking at once, parameters adjustment with GD, repeat E times
- **Mini-batch GD:** randomly select N data points from data, parameters adjustment with GD, repeat T times
- **Stochastic GD:** take each individual data points, parameters adjustment with GD, continue until dataset is covered.

Data	Batch GD	Mini-Batch GD (N=3)	Stochastic GD
x in X: X contains all N in [1, 100]	• Take all N. • Calculate loss. • Adjust with GD. • Repeat E times.	• Take 3 random numbers from N. • Calculate loss. • Adjust with GD. • Repeat T times. • Then repeat E times	• Take one number from N at a time. • Calculate loss. • Adjust with GD. • Grab next number. • complete data set and repeat E times

Problems - Vanishing and exploding gradient

As the network grows, so does the chain rule multiplication necessary for optimizing parameters.

As $N \rightarrow \infty$ (or a big, big number), if:

1. terms in chain rule > 1 , gradient could explode (increase in size such that it will never converge).
2. terms in chain rule < 1 and > 0 , then adjustment will decrease until change is no longer possible (algorithm will never reach the optimal point).

Problems - Hyperparameter tuning

Remember hyperparameters are not adjusted, they are selected by the Engineer/programmer.

We can use validation techniques to assess the best hyperparameter values.

$$\theta_{new} = \theta_{old} - \alpha \nabla_{\theta} L$$

How can we use validation and cross validation? (think)

Problems - Hyperparameter tuning

Hand tuning



Bayesian optimization

(update belief of best
hyperparameter based on pairs
(theta, y))

Random search

(usually: uniform PDF)



Population-based training

(genetic algorithm)



$$\theta_{new} = \theta_{old} - \alpha \nabla_{\theta} L$$

References

1. Russell, S. and Norvig, P. (2020). Artificial intelligence. A Modern Approach. Pearson.
2. Geron, A. (2019) Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems. 2nd Edition, O'Reilly Media, Inc.
3. Manndsylva. (2024, August 10th). From theory to practice: Comparing the McCulloch-Pitts and Rosenblatt perceptron models. Medium.
<https://medium.com/@manndsylva/from-theory-to-practice-comparing-the-mcculloch-pitts-and-rosenblatt-perceptron-models-d5c01da507e9>